Deal Sniper — Architecture Review Report

Scope: `dealscanner/dealbot`

Date: 2026-05-28

Constraint: Read-only analysis; no code changes.

Executive Summary

Deal Sniper is a single-process Node.js application with a linear scan pipeline orchestrated by `src/scan.js`. Persistent state is spread across four actively used JSON/JSONL stores plus two legacy paths. The system works well as a proof-of-concept, but **observations, identity, market stats, watch configuration, and alert deduplication are largely co-located in memory and denormalized into the price-history log**.

Recent additions (`productKey`, dedupe, alert-state keyed by product) move toward clearer product identity, but baselines remain watch-scoped, configuration is stored in a file named `products.json`, and `scan.js` couples scraping, validation, identity, market intelligence, alerting, and persistence in one loop.

---

1. Persistent JSON / JSONL Files

The application uses **four active persistence locations** (plus two legacy paths and one manual backup not referenced by code).

File	Active	Gitignored	Format
`data/products.json`	Yes	No (committed)	JSON array
`data/baselines.json`	Yes	Yes	JSON object map
`data/alert-state.json`	Yes	Yes	JSON object map
`data/price-history-YYYY-MM.jsonl`	Yes	Yes	JSONL (one JSON object per line)
`data/price-history.jsonl`	Legacy (read-only reference)	Yes	JSONL
`data/baselines.old.json`	**Not used by code**	No	JSON object map (manual backup)

Logs (`logs/dealsniper-YYYY-MM.log`) are plain text, not JSON, and are excluded from this persistence review except where they mirror operational events.

---

2. Per-File Analysis

2.1 `data/products.json`

Purpose: User watch configuration — defines what to scrape, how to filter results, and alert thresholds.

Schema:

Field	Type	Required	Description	
(root)	`Watch[]`	—	Array of watch definitions	
`name`	`string`	Yes	Human-readable label; also used as a **foreign key** in baselines, history, and alert state	
`store`	`string`	Yes	Store adapter id (currently `"newegg"`)	
`url`	`string`	Yes	Search/results URL to scrape	
`targetPrice`	`` `number | ``	omitted`	No	Alert when candidate price ≤ this value (USD)
`requirements`	`` `Requirements | ``	omitted`	No	Title-based filters before candidate selection
`requirements.generation`	`string`	No	Required substring in title	
`requirements.totalCapacityGB`	`number`	No	Capacity word match (e.g. 32 → 32GB)	
`requirements.allowedKitLayouts`	`string[]`	No	Kit layout patterns (e.g. `"2x16"`)	
`requirements.excludeTerms`	`string[]`	No	Reject if any term appears in title	

Example record:

```
{
  "name": "DDR5 32GB Newegg Test",
  "store": "newegg",
  "url": "https://www.newegg.com/p/pl?d=DDR5+32GB+6000",
  "targetPrice": 80,
  "requirements": {
    "generation": "DDR5",
    "totalCapacityGB": 32,
    "allowedKitLayouts": ["2x16"],
    "excludeTerms": ["SO-DIMM", "SODIMM", "Laptop", "Mac", "Server"]
  }
}
```

Readers:

Module	Usage
`src/scan.js`	Loads entire array at scan start (`fs.readFileSync`)

Writers:

Module	Usage
(none)	Edited manually; no runtime writes

2.2 `data/baselines.json`

Purpose: Rolling price statistics used for anomaly detection (`price ≤ baseline.averagePrice * 0.55` when `marketSampleSize >= 10`). Despite README language about "category average," this file tracks **per-watch candidate price history**, not category or product market indices.

Schema:

Level	Field	Type	Description
Root	`{ [baselineKey: string]: BaselineEntry }`	object	Map keyed by `store:watchName`
Entry	`averagePrice`	`number`	Running mean of watch candidate prices
Entry	`marketSampleSize`	`number`	Count of candidate observations incorporated
Entry	`lowestSeen`	`number`	Min candidate price seen
Entry	`highestSeen`	`number`	Max candidate price seen
Entry	`updatedAt`	`string` (ISO 8601)	Timestamp of last update

Baseline key format: `{store}:{watchName}` — e.g. `"newegg:DDR5 32GB Newegg Test"`

Example record:

```
{
  "newegg:DDR5 32GB Newegg Test": {
    "averagePrice": 369.99,
    "marketSampleSize": 1,
    "lowestSeen": 369.99,
    "highestSeen": 369.99,
    "updatedAt": "2026-05-25T23:08:32.625Z"
  }
}
```

Readers:

Module	Usage
`src/baselines.js`	`readBaselines()`, `getBaseline()`
`src/scan.js`	Via `getBaseline()` before alert evaluation
`src/status.js`	Displays summary at end of `npm run status`

Writers:

Module	Usage
`src/baselines.js`	`updateBaseline()` — full-file read-modify-write on each watch candidate update
`src/scan.js`	Invokes `updateBaseline(candidate)` once per watch per scan

2.3 `data/alert-state.json`

Purpose: Telegram alert deduplication — stores last alert time and price per logical alert identity so repeated scans do not spam the user.

Schema:

Level	Field	Type	Description
Root	`{ [alertStateKey: string]: AlertStateEntry }`	object	Map of prior alerts
Entry	`lastAlertedAt`	`string` (ISO 8601)	When Telegram last sent for this key
Entry	`lastAlertedPrice`	`number`	Price at last sent alert

Alert state key format:

- Preferred: `{store}:{watchName}:{productKey}` when `productKey` exists
- Fallback: `{store}:{watchName}:{url}` (legacy listings without identity)

Example record (derived from code; file is created on first alert send):

```
{
  "newegg:DDR5 32GB Newegg Test:newegg:item:N82E16820982308": {
    "lastAlertedAt": "2026-05-28T12:00:00.000Z",
    "lastAlertedPrice": 75.99
  }
}
```

Readers:

Module	Usage
`src/alert-state.js`	`readAlertState()`, `shouldSendTelegramAlert()`

Writers:

Module	Usage
`src/alert-state.js`	`recordAlertSent()` — full-file read-modify-write after successful Telegram send
`src/scan.js`	Invokes `recordAlertSent(candidate)` when alert is sent

Note: At review time, `alert-state.json` may not exist until the first alert is sent; `readAlertState()` returns `{}` if missing.

2.4 `data/price-history-YYYY-MM.jsonl`

Purpose: Append-only audit log of **every scraped listing row** per scan, enriched with validation, identity, dedupe metadata, watch context, baseline snapshot, and alert evaluation outcomes.

Schema: Each line is one JSON object. Fields vary by evolution era (pre/post identity enrichment). Current full shape:

Field	Type	Always present	Source concern
`checkedAt`	ISO string	Yes	Observation timestamp
`watchName`	string	Yes	User watch config
`store`	string	Yes	Vendor
`title`	string	Yes	Listing
`url`	string	Yes	Listing
`price`	number | null	Yes	Listing
`targetPrice`	number | null	Yes	**Watch config** (copied onto every row)
`shippingText`	string	Yes	Listing
`validationPassed`	boolean	Yes	Validation
`validationReasons`	string[]	Yes	Validation
`neweggItemId`	string	Newegg only	Product identity
`modelNumber`	string	Newegg only	Product identity
`normalizedUrl`	string	Newegg only	Product identity
`productKey`	string	Newegg only	Product identity
`productKeySource`	string	Newegg only	Product identity
`identityConfidence`	string	Newegg only	Product identity
`dedupeGroupKey`	string	When dedupe applies	Dedupe
`dedupeRole`	`"kept"` | `"duplicate"` | `"not_applicable"`	Post-dedupe	Dedupe
`isWatchCandidate`	boolean	Yes	Candidate selection
`baselineAverage`	number | null	Yes	**Market intelligence snapshot**
`marketSampleSize`	number	Yes	**Market intelligence snapshot**
`alertStateKey`	string	Candidate rows only	Alert identity

`alertStateKeySource`	`"productKey"` \	`"url"`	Candidate rows only	Alert identity
`alert`	boolean	Yes	Alert rule evaluation	
`telegramSent`	boolean	When `alert === true`	Alert dispatch outcome	

Readers:

Module	Usage
(none in application code)	Write-only from the app's perspective; intended for human/ops review
`src/status.js`	Reports file size/mtime only

Writers:

Module	Usage
`src/scan.js`	`fs.appendFileSync(priceHistoryPath(root), ...)` — one line per listing per scan

Path resolution: `src/monthly-paths.js` → `data/price-history-{YYYY-MM}.jsonl`

2.5 `data/price-history.jsonl` (legacy)

Purpose: Pre-rotation append-only history. **No longer written.**

Readers: `src/status.js` lists it under "Legacy files" if present.

Writers: None (deprecated in favor of monthly files per README).

2.6 `data/baselines.old.json` (not application data)

Purpose: Appears to be a **manual backup** of an earlier baseline snapshot. Not referenced by any module.

Recommendation for operators: Treat as archival; exclude from architectural contracts.

---

3. Observation Lifecycle: Scrape → Alert

The following traces one listing through a single watch during `runScan()` in `src/scan.js`.

Sequence (text diagram)

1. **scan.js** reads `products.json` (watches)

2. **stores/newegg.js** scrapes listings from watch URL

3. **validation.js** validates each title against `requirements`

4. **identity/newegg-ram.js** enriches with `productKey`, model number, etc.

5. **dedupe-by-product-key.js** collapses duplicate products; annotates rows

6. **scan.js** picks lowest-price valid listing as **candidate**

7. **baselines.js** reads prior baseline (before update), then updates from candidate

8. For each listing row:

- Attach baseline snapshot, candidate flag, alert evaluation

- If candidate and alert fires: **alert-state.js** dedupe check → **telegram.js** send → record alert

- Append row to **price-history JSONL**

Stage-by-stage detail

Step	Location	Input	Output / side effect
1. Load watches	`scan.js:72`	`products.json`	In-memory `products[]`
2. Scrape	`stores/newegg.js`	Watch `url`	Up to 10 listings
3. Validate	`validation.js`	Title + `requirements`	`validationPassed`, `validationReasons`
4. Identity enrich	`identity/newegg-ram.js`	`url`, `title`	`productKey`, identity fields
5. Dedupe	`dedupe-by-product-key.js`	Valid listings	Collapsed candidate pool
6. Candidate pick	`scan.js:pickWatchCandidate`	Deduped valid listings	Single lowest-price listing
7. Baseline read/write	`baselines.js`	Candidate price	Updates `baselines.json`
8. Per-row enrichment	`scan.js:144–189`	Each listing	Denormalized fields
9. Alert rules	`scan.js:shouldAlert`	Candidate + baseline before update	Boolean alert condition
10. Alert dedupe	`alert-state.js`	Candidate	Cooldown / price-improvement gate
11. Telegram	`telegram.js`	Message	External API call
12. Persist observation	`scan.js:191`	Full enriched row	Append to monthly JSONL

Alert rule logic (current)

Alert when **either**:

- `price <= targetPrice`, or
- `marketSampleSize >= 10` AND `price <= baseline.averagePrice * 0.55`

Only the **watch candidate** row is evaluated for alerts.

---

4. Mixed Concerns

Concern	Primary storage	Also embedded in	Coupling risk
User watch configuration	`products.json`	Every history row	Renaming watch breaks baseline keys
Listing (vendor offer)	*(no registry)*	History rows	No stable listing id separate from URL
Product identity	*(no registry)*	History rows; alert-state keys	Vendor-prefixed strings only
Observation (price snapshot)	`price-history-*.jsonl`	Same row carries derived fields	Log is not a pure fact table
Market / category intelligence	`baselines.json`	History rows	"Market" is watch-candidate average
Alert history / dedupe	`alert-state.json`	History rows	No immutable alert event record

Primary mixing hotspots

1. **`src/scan.js`** — orchestration monolith coupling all domains
2. **Price-history JSONL** — denormalized wide rows (facts + derived metadata)
3. **Baselines keyed by watch name** — market intelligence conflated with user watch
4. **Alert state vs alert events** — dedupe cursor only; no append-only alert log
5. **Identity in scan.js** — not behind store-agnostic port

---

5. Misleading or Drifted Names

Name	Location	Implied meaning	Actual behavior
`products.json`	`data/`	Product catalog	**Watch list**
`product`	`scan.js` loop	Product entity	**Watch** config object
`marketSampleSize`	baselines, history	Category sample size	**Watch candidate** sample count
`baselineAverage`	History rows	Market average	**Watch-level** rolling mean
`averagePrice`	`baselines.json`	Generic average	Average of **candidate** prices only
`isWatchCandidate`	History rows	Generic candidate	**Lowest valid deduped listing** this scan
`alert`	History rows	Alert was sent	Alert **condition met**; send may be suppressed
`newegg-ram.js`	`src/identity/`	RAM-specific	**Generic Newegg** identity
`dedupeGroupKey`	History rows	Abstract group id	Equals `productKey` when present

---

6. Future Scalability Risks

6.1 Multiple stores

- `productKey` format embeds vendor in string; no cross-vendor canonical id
- Newegg-specific history fields require parallel columns per vendor
- Identity enrichment hardcoded in `scan.js`
- Baseline and alert keys scoped to watch + store, not product globally

6.2 Broad product categories

- `requirements` and `validation.js` are RAM-specific, not pluggable
- No `categoryId` on watches
- Baselines per watch cannot represent category-wide trends without aggregation

6.3 Historical price tracking expansion

- Monthly JSONL requires external tooling for cross-month queries

- Heterogeneous row shapes (schema evolution)
- Denormalized baseline on every row
- No `productId` / `listingId` foreign keys
- `isWatchCandidate` via URL equality breaks on URL changes
- No observation id or scan id for idempotent processing

---

7. Proposed Target Architecture

7.1 Product

Canonical identity independent of vendor listings.

- `productId`, `canonicalKey`, `categoryId`, `attributes`, `vendorKeys`, timestamps

Storage: `data/products-registry.json` (distinct from watch config)

7.2 Listing

Vendor-specific offer tied to a product.

- `listingId`, `productId`, `store`, `vendorListingId`, `url`, `title`, lifecycle timestamps

7.3 Observation

Immutable point-in-time price measurement.

- `observationId`, `scanId`, `observedAt`, `watchId`, `listingId`, `productId`, `price`, validation fields

Storage: Append-only `data/observations-YYYY-MM.jsonl` — **facts only**

7.4 Category intelligence

Derived market statistics, separate from watches.

- `categoryId`, rolling stats (`avg`, `median`, sample size), `window`, `computedAt`

Optional parallel: `WatchCandidateStats`, `ProductPriceStats`

7.5 User watch

User monitoring intent and alert rules.

- `watchId`, `name`, `store`, `url`, `categoryId`, `filterProfile`, `alertRules`, `enabled`

Storage: `data/watches.json`

7.6 Alert event

Immutable alert evaluation and dispatch record.

- `alertEventId`, `watchId`, `productId`, `listingId`, `observationId`, `ruleMatched`, `dispatch`

Dedupe state (mutable): separate `data/alert-dedupe.json`

---

8. Entity Relationship Diagram

```
USER_WATCH ──produces──> OBSERVATION <──priced at── LISTING
                              │                        │
                              │                        └── belongs to ──> PRODUCT
                              │                                              │
                              └── may trigger ──> ALERT_EVENT                └── classified in
──> CATEGORY
                                        │                                          │
                                        └── updates ──> ALERT_DEDUPE_STATE         └──
aggregated into ──> CATEGORY_INTELLIGENCE

PRODUCT ──> PRODUCT_STATS (derived)
USER_WATCH ──> WATCH_CANDIDATE_STATS (derived, current baselines.json behavior)
```

Dependency direction:

```
User Watch → Scan → Observation → (derive) → Intelligence
                              ↘ (evaluate) → Alert Event → Alert Dedupe State
Product Registry ← Identity resolution ← Listing scrape
```

---

9. Recommended Migration Path

Phase 0 — Document and freeze

Treat `products.json` as watches; document jsonl schema versions.

Phase 1 — Persistence and pipeline boundaries

Repository modules; split `runScan()` stages; candidate match by `productKey`.

Phase 2 — Stable watch identity

Add `watchId`; dual-write baseline keys; rename to `watches.json` with fallback.

Phase 3 — Slim observations

Facts-only observation schema; move derived fields to alert events / scan summary.

Phase 4 — Product registry

Introduce `products-registry.json`; add `productId` to observations and dedupe keys.

Phase 5 — Category model

`categoryId`, pluggable filters, `category-stats.json` rebuilt from observations.

Phase 6 — Alert events

Append `alert-events-*.jsonl`; reduce `alert-state.json` to dedupe cursor.

Phase 7 — Second vendor

Store adapter + identity module behind interfaces.

Phase 8 — Historical analytics

Optional SQLite/DuckDB index over observation jsonl.

Migration principles

1. Dual-write, then cut over

2. Append-only observations with `schemaVersion`

3. Derived state is rebuildable

4. Correctness over speed for cross-vendor linking

5. No big-bang rewrite

Appendix: Module ↔ Persistence Matrix

Module	products.json	baselines.json	alert-state.json	price-history JSONL
`scan.js`	R	R/W	R/W	W
`baselines.js`	—	R/W	—	—
`alert-state.js`	—	—	R/W	—
`status.js`	—	R	—	metadata only
`watch.js`	—	—	—	—
`index.js`	—	—	—	—

Summary

Deal Sniper's persistence layer is small and operable, but **conceptually overloaded**: the price-history JSONL acts as observation log, debug trace, market snapshot, and partial alert record simultaneously; baselines represent watch-candidate trends under market naming; and `products.json` stores watches, not products.

The recent identity and dedupe work establishes a foundation for product-aware behavior. The highest-value architectural next step is **separating immutable observations from derived intelligence and alert outcomes**, introducing stable `watchId` and `productId` foreign keys, and wrapping file I/O in repositories — all achievable without disrupting the current Newegg RAM monitoring workflow.